HW 8 due this Friday.

No Quiz this week.

Class 24: Reductions, Class NP

University of Virginia
CS3120: DMT2
https://weikailin.github.io/cs3120-toc
Wei-Kai Lin

Plan

Polynomial-time Reductions

Textbook [TCS] Section 13

https://introtcs.org/public/lec_11_running_time.html
Modeling running time

Textbook [TCS] Section 14

https://introtcs.org/public/lec_12_NP.html
Polynomial-time reductions

Gemini generated cover image.

Recap

Complexity Classes: P and EXP

"Polynomial Time" and "Exponential Time"

Definition 13.2, Textbook

Class **P** = $\bigcup_{c \in \mathbb{N}} TIME_{TM}(n^c)$

Class **EXP** = $\bigcup_{c \in \mathbb{N}} TIME_{TM}(2^{n^c})$

P $\subset$ **EXP** , strict

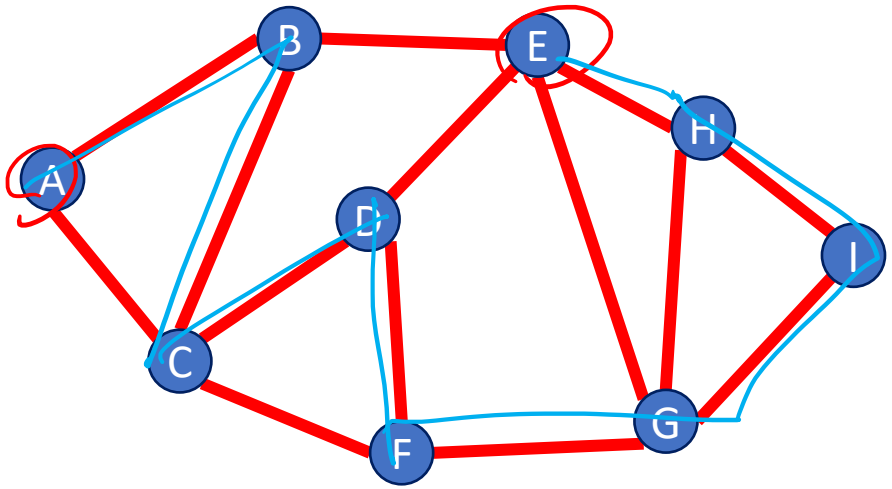
Robust: Same set for TM, RAM, Python, …

Definition: a ***path*** from v_1 to v_k in a graph $G = (V, E)$ is a sequence of nodes $(v_1, v_2, \dots, v_k)$ such that $\forall\ 1 \leq i \leq k-1$, $(v_i, v_{i+1}) \in E$. A ***simple path*** is a path with no repeated nodes.

Recap: Longest Path (Search)

Given a start node s and an end node t, how long is longest *simple* path from s to t?

LongestPath-Search(G, s, t)
Input: A finite graph $G = (V, E)$, two vertices, $s, t \in V$
Output: The length of the longest simple path from s to t in G.

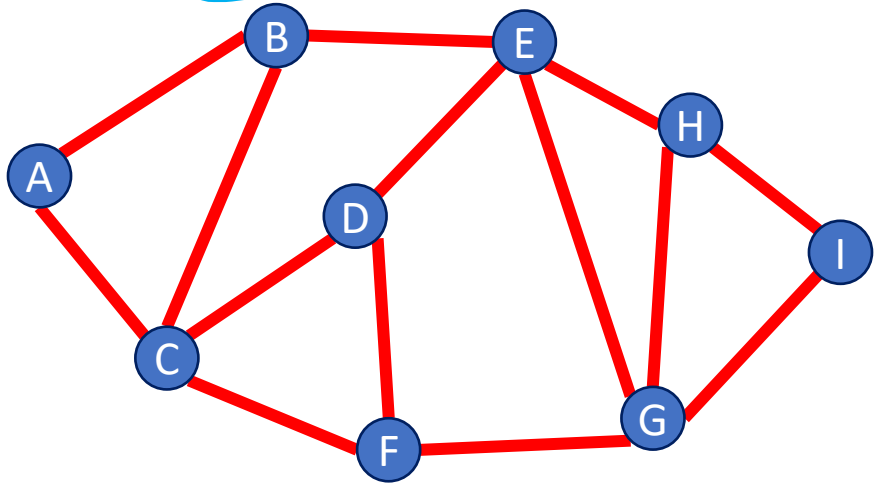


Definition: a ***path*** from v_1 to v_k in a graph $G = (V, E)$ is a sequence of nodes $(v_1, v_2, \ldots, v_k)$ such that $\forall\ 1 \leq i \leq k-1$, $(v_i, v_{i+1}) \in E$. A ***simple path*** is a path with no repeated nodes.

Recap: Longest Path, Decision

Given a start node s and an end node t, how long is longest *simple* path from s to t?

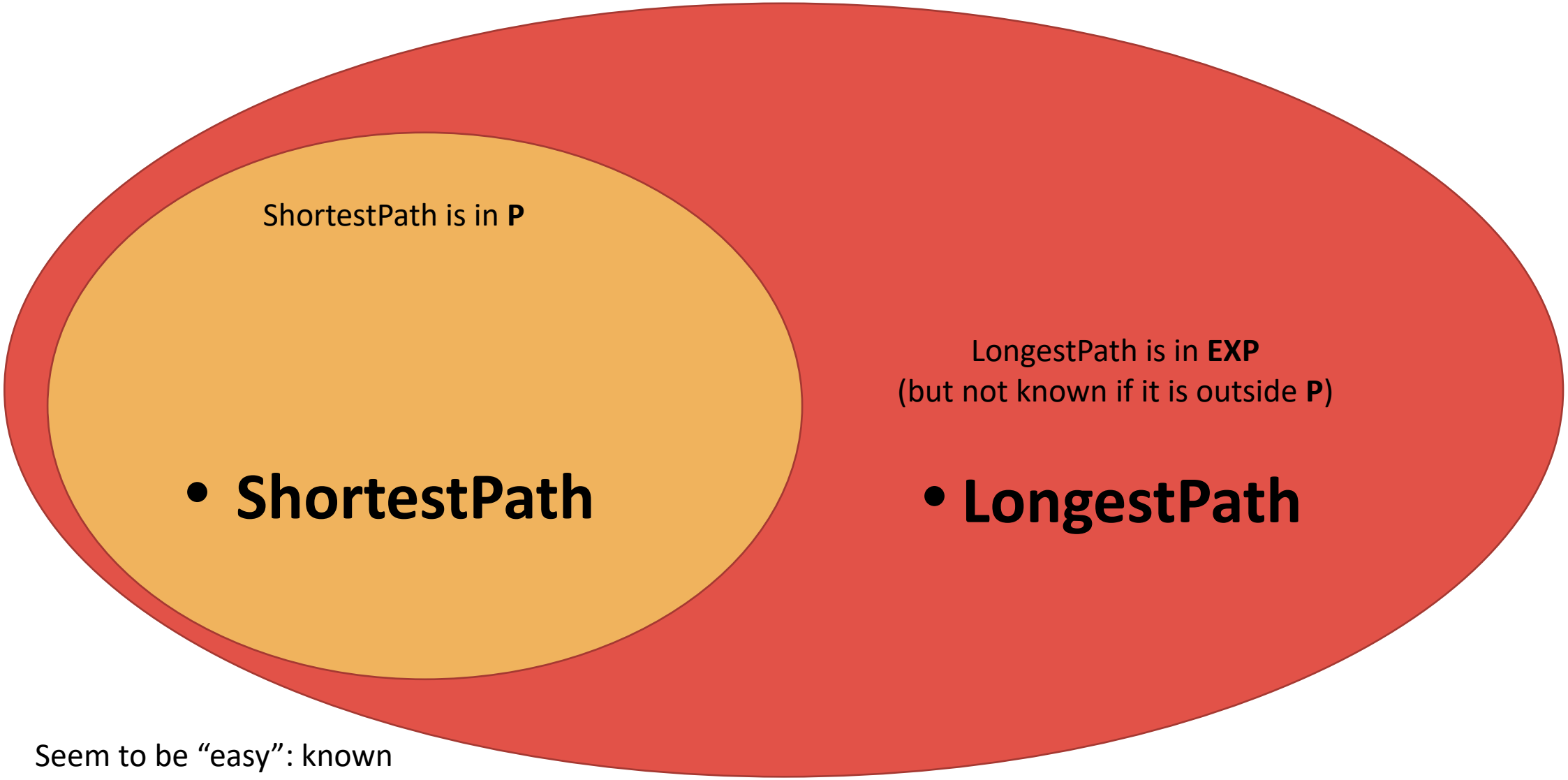
LongestPath-Search(G, s, t)
Input: A finite graph $G = (V, E)$, two vertices, $s, t \in V$
Output: The length of the longest simple path from s to t in G.



LongestPath-Decision (G, s, t, ℓ)
Input: A finite graph $G = (V, E)$, two vertices, $s, t \in V$, and a path length, $\ell \in \mathbb{N}$.
Output: If there is a simple path from s to t in G of length at least ℓ, 1. Otherwise, 0.

1 bit

Class EXP:
Exponential Time



Seem to be "easy": known polynomial-time algorithms

Class **P** = $\bigcup_{c \in \{1,2,3,\ldots\}} TIME_{TM}(n^c)$

Class **EXP** = $\bigcup_{c \in \{1,2,3,\ldots\}} TIME_{TM}(2^{n^c})$

Polynomial-Time Reductions

Recap: Computability Reductions

$$A \leq_R B$$

A reduces to B

We can prove a problem **B** is uncomputable by showing that if we could compute **B** we could use the machine that computes **B** to compute **A**, which we already know is uncomputable.

Since we were showing ***uncomputability***, the reduction can do anything that can be **computed by a TM**.

Polynomial-Time Reductions

$$A \leq_p B$$

A (polynomial-time) *reduces* to B

We will use this to prove that problem **B** is as least as "hard" as problem **A** by showing that if computing **B** is in **P** we could use the machine that computes **B** to compute **A** in **P** (but we already know **A** is "hard").

We haven't yet defined "hard" here, or shown any problem we know is "hard" (preview: *LongestPath* is "hard").

Since we are showing a hardness property related to **P**, the reduction must only involve computation that can be done in **P**.

Comparison

$$A \leq_R B$$

A reduces to B

$$A \leq_p B$$

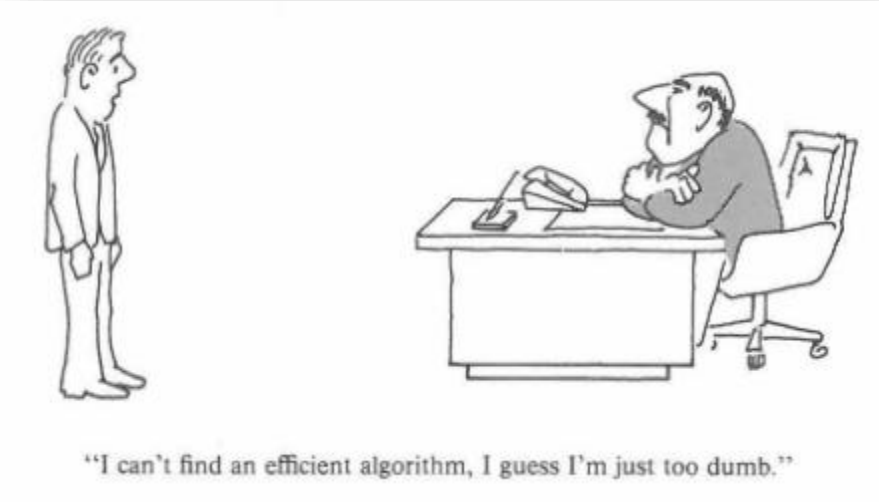
A (polynomial-time) *reduces* to B

Prove non-computability of **B**

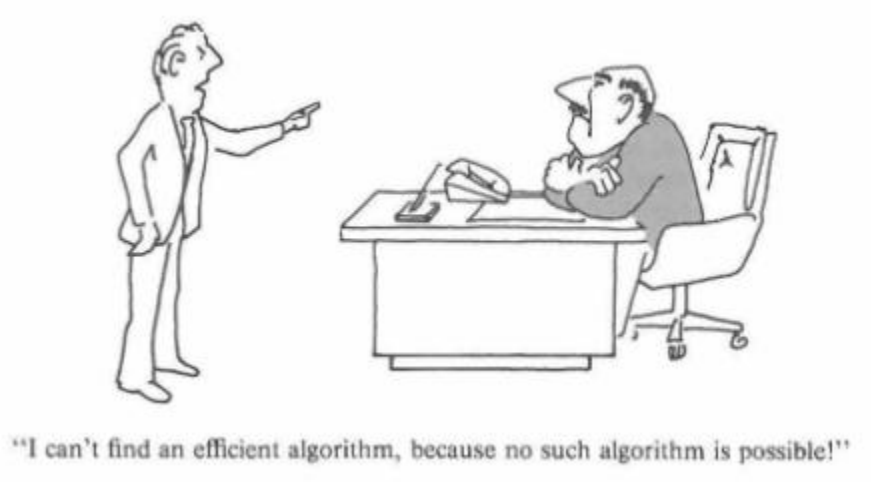
Reduction must be computable

Prove non-"easiness" of **B**

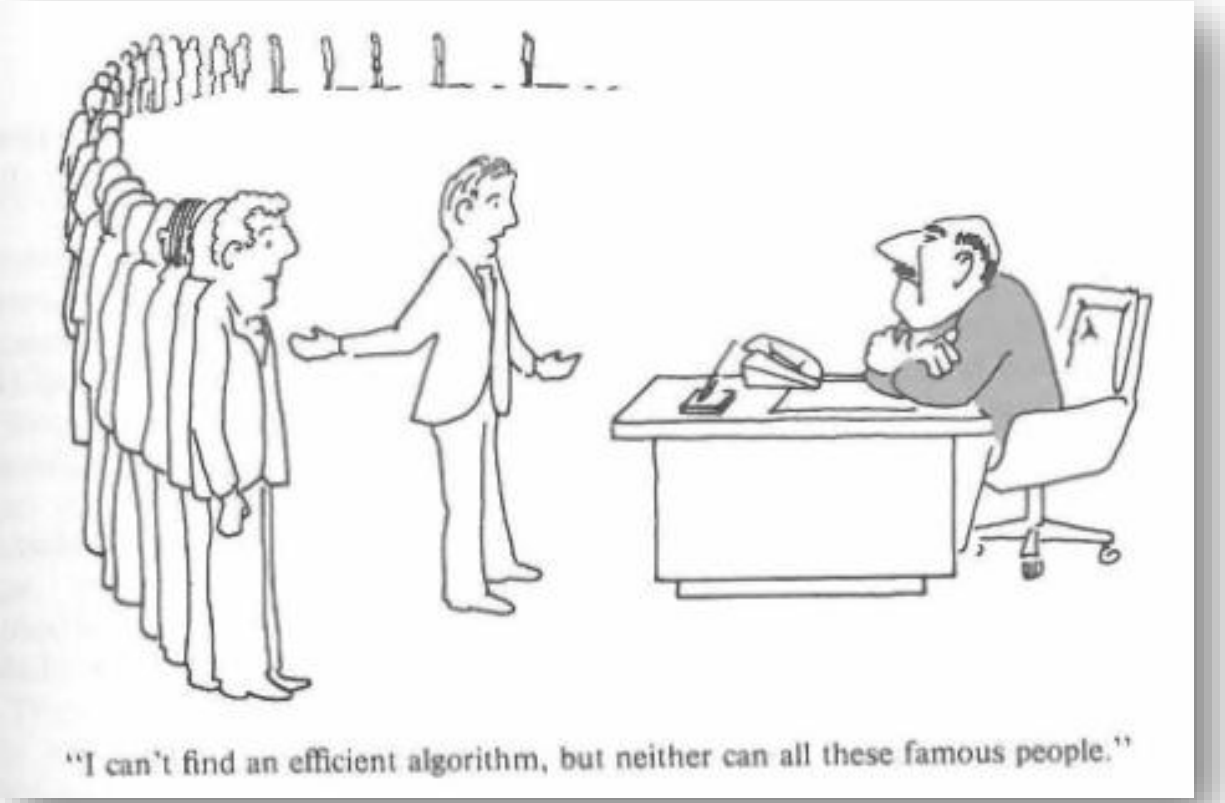
Reduction must be "easy" (in **P**)



"I can't find an efficient algorithm, I guess I'm just too dumb."



"I can't find an efficient algorithm, because no such algorithm is possible!"



"I can't find an efficient algorithm, but neither can all these famous people."

Computers and Intractability: A Guide to the Theory of NP-Completeness.
Michael R. Garey and David S. Johnson

Polynomial Reduction Definition

Definition 14.1 (Polynomial-time reductions)

Let $F, G : \{0,1\}^* \to \{0,1\}$. We say that F *reduces to* G, denoted by $F \leq_p G$ if there is a polynomial-time computable $R : \{0,1\}^* \to \{0,1\}^*$ such that for every $x \in \{0,1\}^*$,

$$F(x) = G(R(x)) \ . \quad (14.1)$$

We say that F and G have *equivalent complexity* if $F \leq_p G$ and $G \leq_p F$.

Note: This is the "Karp reduction" definition. (Cook reduction is more general)

No cooking!

Stephen Cook

OC OOnt

Cook in 2008

WIKIPEDIA 25

25 years of the free encyclopedia

Donate Create account Log in

Reduction (cooking)

19 languages

Article Talk Read Edit View history Tools

From Wikipedia, the free encyclopedia

oking, **reduction** is the process of thickening and intensifying the flavor of a liquid mixture, such as a soup, sauce, wine or juice, by simmering or boiling.[1]

Reduction is performed by simmering or boiling a liquid, such as a stock, fruit or vegetable juice, wine, vinegar or sauce, until the desired concentration is reached by evaporation. This is done without a lid, enabling the vapor to escape from the mixture. Different components

Stock being reduced in a pan

https://en.wikipedia.org/wiki/Stephen_Cook

Trivial Example

1 bit output

$F, G\colon \{0,1\}^* \to \{0,1\}$. $F \leq_p G$ if there is a polynomial-time computable $R\colon \{0,1\}^* \to \{0,1\}^*$ such that for every $x \in \{0,1\}^*, F(x) = G(R(x))$.

Prove

F G

$$LongestPathDecision \leq_p LongestPathDecision.$$

$F(x) = G(R(x))$

$R(x)$:

1. Output x

How about proving:

$$LongestPathDecision \leq_p LongestPathSearch?$$

F G

Less Trivial Example

F, G: $\{0,1\}^* \to \{0,1\}$. $F \leq_p G$ if there is a polynomial-time computable R: $\{0,1\}^* \to \{0,1\}^*$ such that for every $x \in \{0,1\}^*$, $F(x) = G(R(x))$.

Prove $\underset{F}{ODD} \leq_p \underset{G}{EVEN}$.

ODD: $\{0,1\}^* \to \{0,1\}$. Output 1 of input is binary representation of a odd natural number, 0 otherwise.
$EVEN$: $\{0,1\}^* \to \{0,1\}$. Output 1 of input is binary representation of a even natural number, 0 otherwise.

$R(x)$

1. $y := x+1$
2. Output y

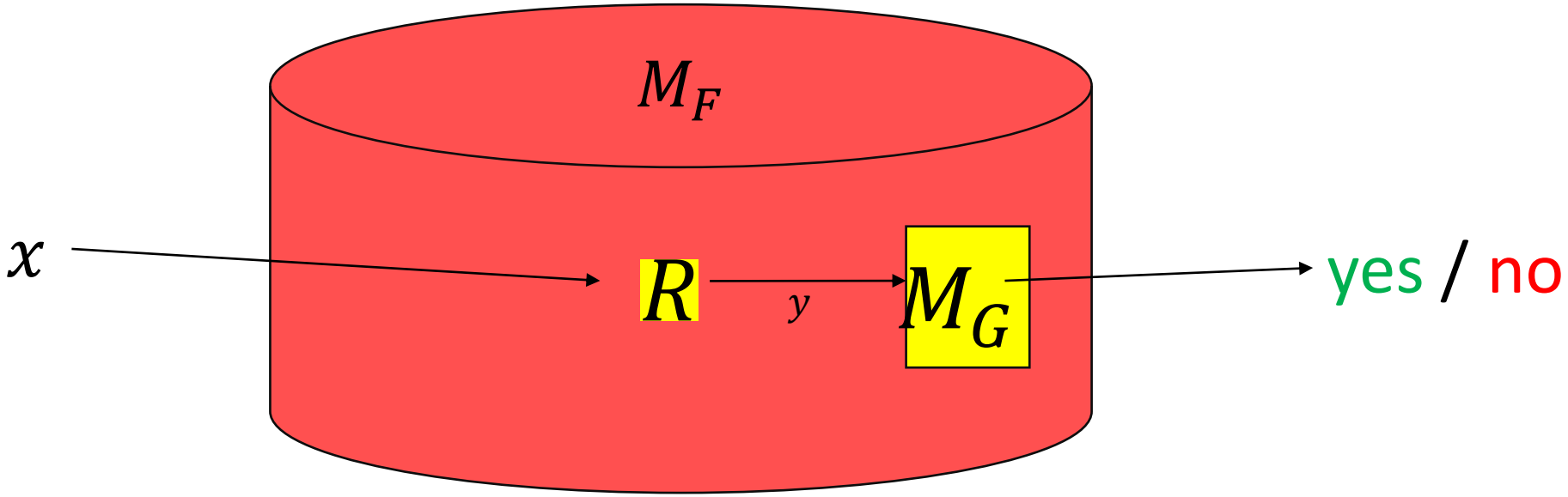
More poly-time reductions

The picture for Karp reductions

We have a solver M_G for G F , G(x) = NOT(F(x))

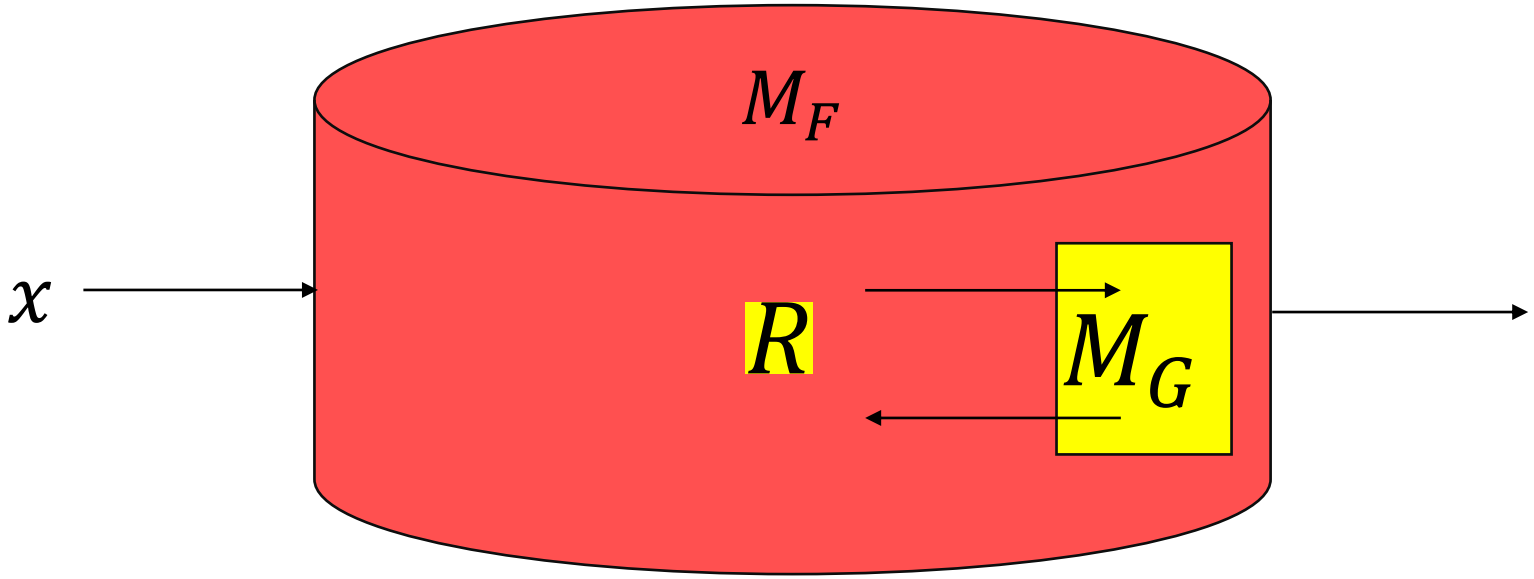
We design a solver M_F for F as follows:

1. Use the poly-time reduction R to modify input x into y
2. Output $M_G(y) = M_G\big(R(x)\big)$



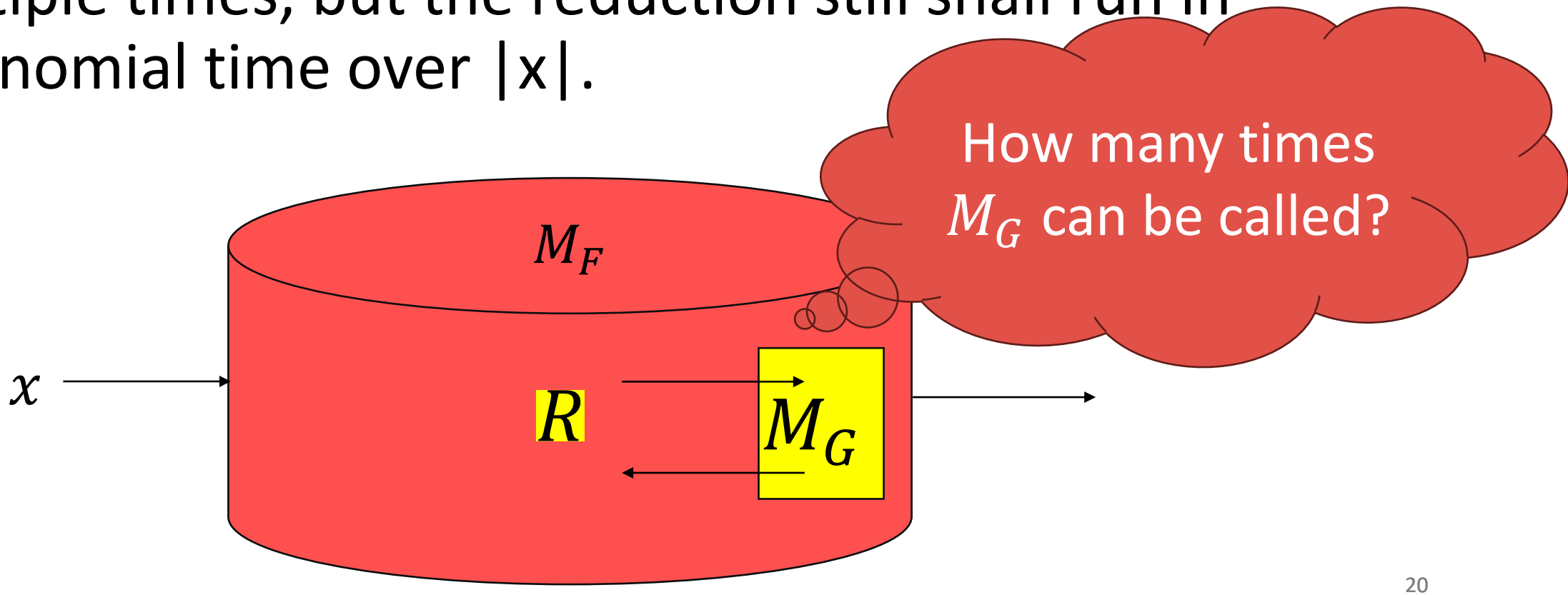
Cook reductions

In Cook reductions, the "subroutine" M_G can be used multiple times, but the reduction still shall run in polynomial time over |x|.



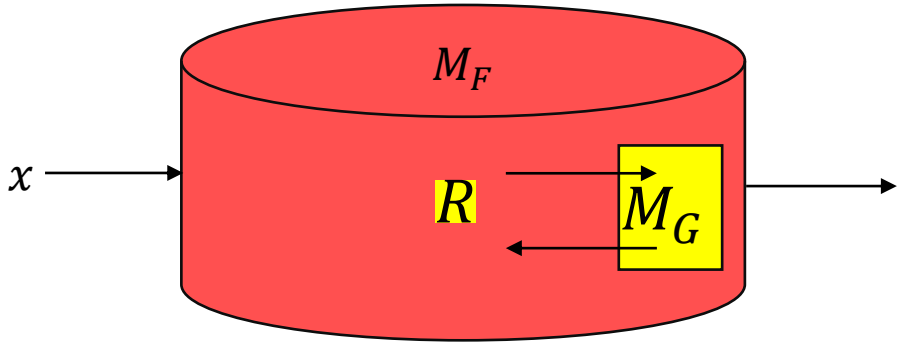
Cook reductions

In Cook reductions, the "subroutine" M_G can be used multiple times, but the reduction still shall run in polynomial time over |x|.

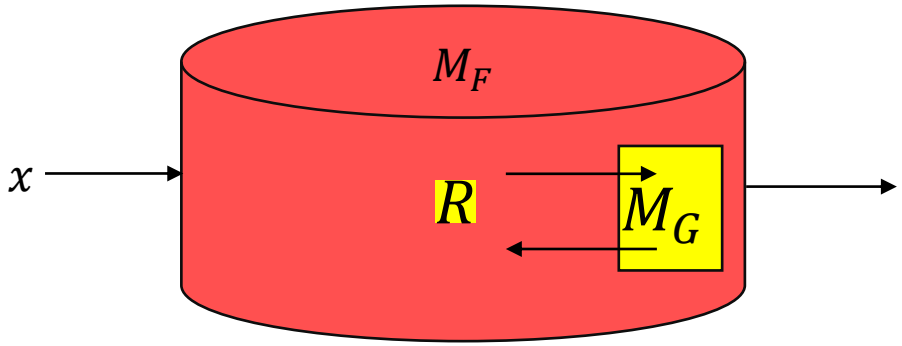


Examples

$LongestPathDecision \leq_p LongestPathSearch.$



$LongestPathSearch \leq_p LongestPathDecision.$



Karp Reductions vs. Cook Reductions

Every Karp reduction is a Cook reduction too.

A Cook reduction is not necessarily a Karp reduction.

Uses of Reductions

Reductions can be used to show easiness and hardness

Suppose $F \leq_p G$ then we have:

- $G \in \mathbf{P} \rightarrow F \in \mathbf{P}$ (deriving easiness)
- $F \notin \mathbf{P} \rightarrow G \notin \mathbf{P}$ (deriving hardness)
  Michael Sipser: "whistling pigs → flying horses"

“Hard” Problems: SAT

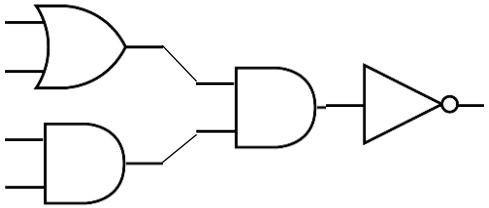
Another Problem (Seemingly) Outside P

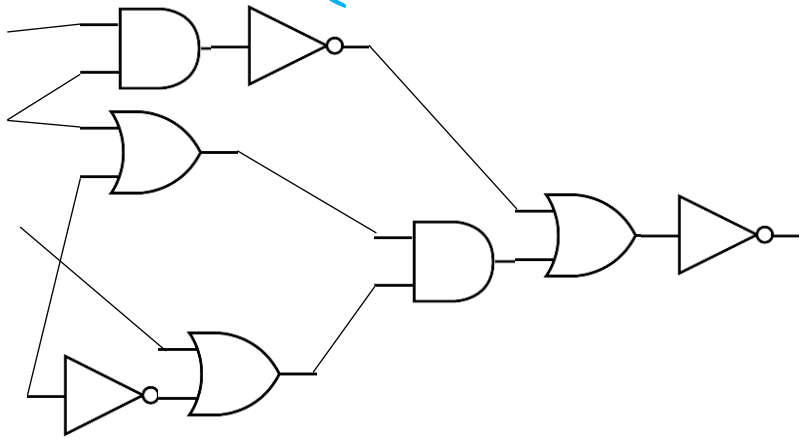
CircuitSAT :
$\{C : C$ is a Boolean circuit such that $\exists x,\ C(x) = 1\}$

Another Problem (Seemingly) Outside P

CircuitSAT :

$\{C : C$ is a Boolean circuit such that $\exists x,\ C(x) = 1\}$





3-CNF (Conjunctive Normal Form)

3-CNF formula is logical AND of clauses, each an OR of 3 literals.

$$F(x_1, x_2, x_3, x_4) =$$

$$\underbrace{(x_1 \vee x_2 \vee x_3)}_{\text{Clause}} \wedge (x_1 \vee \overline{x_2} \vee x_2) \wedge (x_4 \vee x_2 \vee \overline{x_3}) \wedge (x_3 \vee \overline{x_1} \vee x_4) \wedge (\overline{x_1} \vee \overline{x_2} \vee \overline{x_3})$$

Variables

3-CNF

Def: Suppose $x_1, \dots x_n$ are Boolean **variables**.

Any x_i or $\overline{x_i} = 1 - x_i$ is a **literal**.

Any $(z_i \vee z_j \vee z_k)$ over literals z_i, z_j, z_k is a **3-clause**.

A **3-CNF** (conjunctive normal form) formula looks like $C_1 \wedge C_2 \wedge \cdots C_m$ in which all C_i are 3-clauses.

We say assignment x to $x_1, \dots x_n$ satisfies 3-CNF F, denoted by $F(x) = 1$, if at least one literal in each clause is TRUE.

3-SAT (Satisfiability)

Given a 3-CNF formula F, is there an assignment of true/false such that F(assignment)=1?

$$F(x_1, x_2, x_3, x_4) = 1$$
$$(x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_2) \wedge (x_4 \vee x_2 \vee \overline{x_3}) \wedge (x_3 \vee \overline{x_1} \vee x_4) \wedge (\overline{x_1} \vee \overline{x_2} \vee \overline{x_3})$$

$$\boldsymbol{x_1 = true}$$
$$\boldsymbol{x_2 = false}$$
$$\boldsymbol{x_3 = false}$$
$$\boldsymbol{x_4 = true}$$

Definition:

3-SAT(F) = 1 if there exists x such that F(x) = 1

0 otherwise.

Instances of 3-CNF

Are they satisfiable?

- $(x_1 \vee x_2 \vee \bar{x}_3) \wedge (x_3 \vee x_2 \vee x_4) \wedge (x_1 \vee x_2 \vee x_3)$ Yes
- $(x_1 \vee x_1 \vee x_1) \wedge (\bar{x}_1 \vee \bar{x}_1 \vee \bar{x}_1)$ No
- $(x_1 \vee x_2 \vee \bar{x}_3) \wedge (x_3 \vee \bar{x}_2 \vee x_4) \wedge (x_1 \vee x_2 \vee x_3) \wedge (\bar{x}_1 \vee x_3 \vee x_4)$ Yes

3-SAT $\leq_p$ CircuitSAT

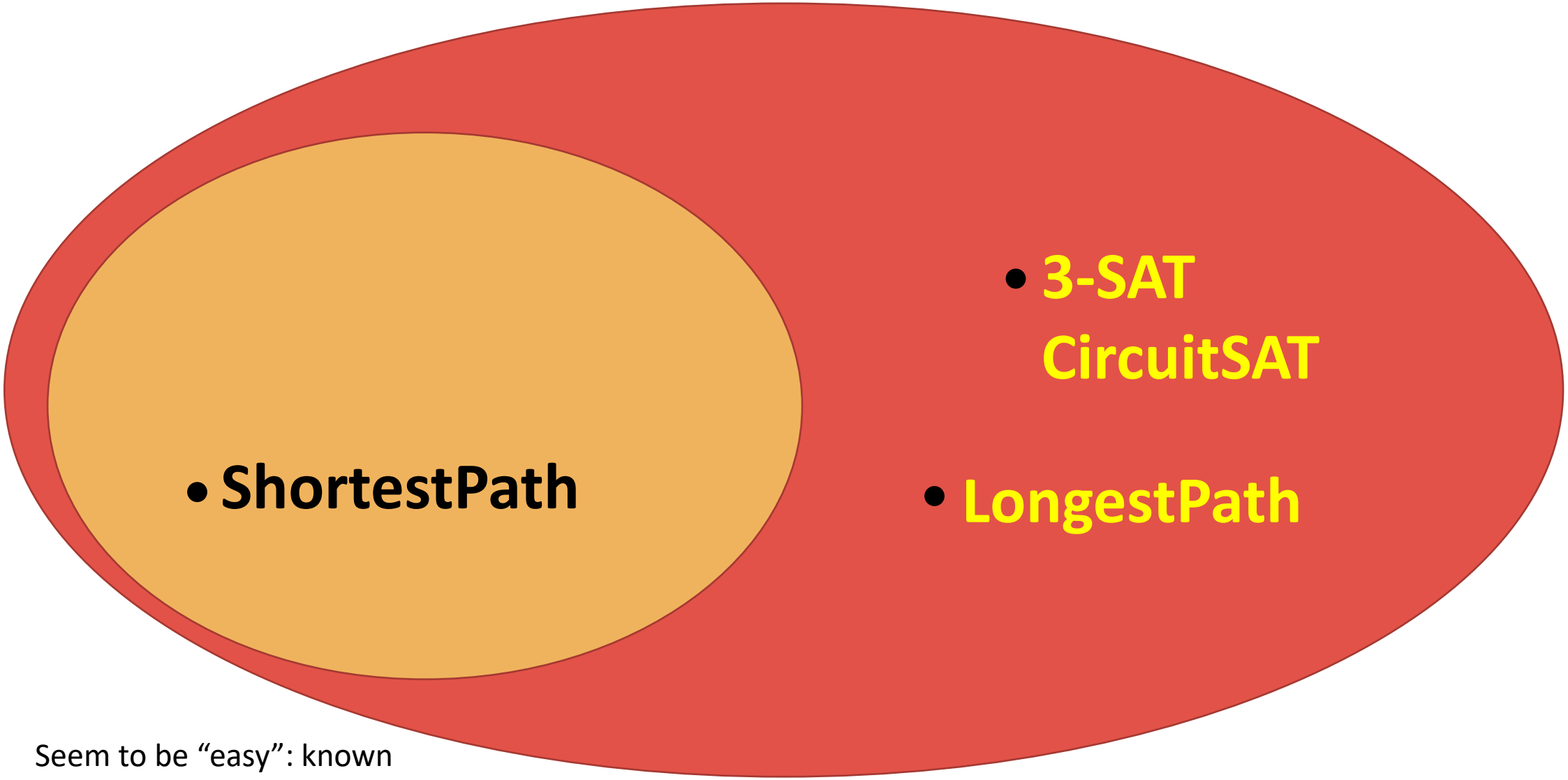
F G

Suppose we are given a 3-CNF formula F. How do we (efficiently) transform F into a circuit C_F such that C_F is satisfiable if and only if F is satisfiable?

$F(x) = G(R(x))$

$R(x)$

1. x a 3-CNF, interpret x as a ckt C_F
2. Output C_F



Seem to be "easy": known polynomial-time algorithms

Class **P** = $\bigcup_{c \in \{1,2,3,\ldots\}} TIME_{TM}(n^c)$

Class **EXP** = $\bigcup_{c \in \{1,2,3,\ldots\}} TIME_{TM}(2^{n^c})$

Logistics

HW 8 due this Friday.

No Quiz this week.

Plan

NP and NP-Complete

Textbook [TCS] Section 14

https://introtcs.org/public/lec_13_Cook_Levin.html
NP, NP Completeness